

Raíz Andina por dentro

Qué se construyó, con qué está hecho y por qué cada pieza está donde está.
Es la demo que sostiene la propuesta comercial: una tienda naturista peruana completa, del catálogo al comprobante electrónico.

23	9 031	222	400	0	1 874
MÓDULOS	LÍNEAS	PRUEBAS	PRODUCTOS	DEPENDENCIAS	DISTRITOS

01 QUÉ ES

Una tienda que funciona de verdad, no una maqueta

Raíz Andina es una tienda naturista completa que corre en una laptop sin conexión. No hay pantallas de mentira: el cliente compra, el stock baja, se emite la boleta, el panel lo ve y el comprador sigue su pedido hasta firmar la recepción.

Existe para una conversación de venta concreta —convencer al dueño de una tienda naturista de que esto no es «una página web»— así que todo lo que se ve tiene que aguantar que le hagan preguntas incómodas delante.

Son **siete páginas**: la tienda, el seguimiento del pedido, el login, el panel interno, el comprobante imprimible, los créditos de las fotos y la hoja de contactos de las ilustraciones.

02 LA RESTRICCIÓN

Cero dependencias, y eso manda sobre todo lo demás

No hay `npm install`, no hay paso de compilación, no hay `node_modules`. La base de datos es `node:sqlite`, el cifrado es `node:crypto`, las pruebas son `node:test`. Todo viene dentro de Node 22.5 o superior.

Tres razones, en orden de peso:

- **Se demuestra sin internet.** En el puesto del mercado la señal se cae. Una demo que necesita descargar algo es una demo que falla justo cuando hay un cliente mirando.
- **Sigue arrancando en dos años.** Un proyecto con cuarenta paquetes se pudre solo: alguno se abandona, otro cambia de versión y deja de instalar. Sin paquetes no hay nada que se pudra.

- **Se puede leer entero.** Nada pasa por una caja negra de otro. Cada línea que corre está en esta carpeta.

El costo es real y hay que decirlo: cosas que un framework regala —enrutado, plantillas, validación— están escritas a mano. Son unas seiscientas líneas de `server.js`. A cambio, el proyecto entero pesa lo que pesa el código.

03 EL MAPA

Los veinte módulos y de qué se ocupa cada uno

Cada archivo tiene un trabajo y no se mete en el de al lado. Los que empiezan por `gen-` son herramientas que se corren una vez y dejan un archivo hecho; no participan cuando la tienda está andando.

<code>server.js</code>	El servidor HTTP sin framework: rutas, sesiones, cabeceras de seguridad, plantillas y el alta de productos.	1 041
<code>gen-catalogo.mjs</code>	Arma los 400 productos combinando 132 insumos reales con las presentaciones que le tocan a cada uno.	889
<code>db.js</code>	Esquema, migraciones idempotentes y carga inicial. Nueve tablas con restricciones a nivel de base.	517
<code>comprobantes.js</code>	Boletas, facturas y notas de crédito: numeración, aritmética del IGV, XML UBL 2.1 y contenido del QR.	510
<code>asesor.js</code>	Empareja el malestar que escribe el cliente con los productos que lo tratan, filtrados por stock.	449
<code>intenciones.js</code>	Quince intenciones de negocio —delivery, horarios, devoluciones, regateo— y la cotización al por mayor.	442
<code>gen-imagenes.mjs</code>	Dibuja las ilustraciones: 24 por producto curado y 84 de respaldo, silueta por color.	253
<code>importar-fotos.mjs</code>	Toma la carpeta de fotos del negocio y las empareja por nombre, tolerando tildes, plurales y erratas.	198
<code>auth.js</code>	Login con script, sesiones en cookie firmada y bloqueo tras varios intentos fallidos.	175
<code>traer-fotos.mjs</code>	Rellena las fotos que falten desde bancos libres. Lo único que necesita internet, y una sola vez.	165
<code>tienda.config.js</code>	El único punto de edición: teléfono, RUC, horarios, zonas de reparto, IGV, correo.	135
<code>ubigeo.js</code>	Valida departamento, provincia y distrito contra el ubigeo del INEI y calcula la zona de reparto.	130

documentos.js	DNI y RUC con dígito verificador módulo 11. Un RUC inventado no pasa.	117
ilustraciones.mjs	Decide qué lámina le toca a cada producto: la silueta la manda la presentación, el color la categoría.	112
catalogos-sunat.js	Los catálogos 01, 02, 03, 05, 06, 07 y 09 de SUNAT, que son los que exige el XML.	102
verificar-imagenes.mjs	Revisa que las 109 ilustraciones estén dentro del lienzo y centradas, sin abrir un navegador.	100
importe-letras.js	«CIENTO TREINTA Y UNO CON 80/100 SOLES». Va en el comprobante y hay que apocopar bien.	93
limites.js	Cuotas por IP con cubeta de fichas, para que nadie pruebe códigos de pedido ajenos en masa.	66
gen-ubigeo.mjs	Compacta el ubigeo del INEI a 26 KB: 25 departamentos, 196 provincias, 1 874 distritos.	67
clave.mjs	Crea y cambia la contraseña del panel desde la línea de comandos.	83

04 DECISIONES

Nueve decisiones que explican el resto del código

El stock no se edita a mano

Solo se mueve por ventas, ingresos y devoluciones. Para corregirlo se registra un ingreso, que deja constancia. Si el stock se pudiera escribir directo, el kardex dejaría de explicar dónde fue cada unidad —que es exactamente lo que el dueño necesita saber.

En el comprobante manda el importe, no el valor de venta

Se cobra un importe exacto; de ahí sale la base imponible, y el IGV se calcula *por diferencia*. Al revés —calcular el IGV y sumarlo— aparecían descuadres de dos céntimos entre el pedido y la boleta. Dos céntimos en un comprobante son un problema con SUNAT.

El asesor pregunta primero de qué se trata

Hay dos capas. La primera reconoce intenciones de negocio: horarios, delivery, devoluciones, mayoreo, regateo. Solo si no es ninguna de esas se pasa a recomendar productos. Antes, un cliente que reclamaba un pedido demorado recibía un catálogo de hierbas.

Un producto solo califica por sus etiquetas o su nombre

El texto libre de la descripción suma puntos, pero nunca alcanza para recomendar. Sin esa regla, «bajar 10 kilos rápido» devolvía Sangre de Grado, porque su descripción decía «cerrarlas rápido».

La boleta la recoge el comprador, no la pide por teléfono

La misma página imprime el comprobante para los dos lados del mostrador. La tienda entra con su sesión; el comprador, con el mismo par de código y últimos cuatro dígitos con que sigue su pedido. Sale del mismo sitio, así que los importes no pueden diferir. El XML no se le entrega: ese es del emisor y de SUNAT.

El panel no vive en la tienda

`/admin.html` y las demás páginas internas no se sirven sin sesión: redirigen al login antes de mandar un solo byte. Un cliente curioso no debe llegar ahí ni por accidente.

Sin scripts en línea, en ninguna página

La política de contenido es `script-src 'self'` sin `unsafe-inline`. Costó sacar todo el JavaScript a archivos y reemplazar los `onerror=` por oyentes delegados, y es lo que hace que un texto inyectado no pueda ejecutarse.

Cada cambio de ficha queda firmado

Qué campo, valor anterior, valor nuevo, quién y cuándo. Un precio que baja sin responsable es el mismo agujero que un ajuste de stock sin responsable.

Un solo archivo para cambiar el negocio

Teléfono, RUC, horario, IGV, zonas de reparto y costos viven en `tienda.config.js`. Cambiar de número de WhatsApp es editar una línea, no buscar en catorce archivos.

05 SEGURIDAD

Lo que protege los datos del cliente

La tienda guarda nombres, teléfonos, direcciones y documentos de identidad. Eso obliga a más que un login.

- **Contraseñas con scrypt** y comparación en tiempo constante; nunca se guardan en claro.
- **Cookies HttpOnly y SameSite=Strict**, que el JavaScript de la página no puede leer.
- **Bloqueo por intentos fallidos** sobre la cuenta, más cuota por IP para el ataque distribuido.
- **Cuotas por IP:** asesor 30 por minuto, pedidos 8 cada 5 minutos, seguimiento y login 20 cada 10 minutos.

- **Seguimiento con dos factores:** el código del pedido más los últimos cuatro dígitos del teléfono. El código solo no basta.
- **Mismo mensaje de error** para código inexistente y teléfono equivocado, para que no se puedan averiguar códigos válidos.
- **El costo de compra no sale al público.** El catálogo abierto entregaba el margen del negocio completo; ahora se seleccionan las columnas.
- **Transacciones** `BEGIN IMMEDIATE` con claves foráneas y restricciones `CHECK`: un pedido entra completo o no entra.

06 PRUEBAS

222 pruebas automáticas, todas verdes

Cada archivo levanta su propio servidor en un puerto libre y con una base de datos desechable, así que no dependen del orden en que corran ni ensucian la demo. Se ejecutan con `npm test` en unos ocho segundos.

ARCHIVO	QUÉ VIGILA	PRUEBAS
<code>comprobantes</code>	Numeración sin huecos, IGV cuadrado, XML, QR, PDF, notas de crédito	39
<code>checkout</code>	DNI, RUC, ubigeo, costo de envío, correo y teléfono	36
<code>pedidos</code>	Stock, transacciones, estados, devoluciones, códigos repetidos	34
<code>alta</code>	Alta de productos y de categorías nuevas, y lo que debe rechazar	25
<code>auth</code>	Sesiones, bloqueo, cierre de sesión, páginas privadas	25
<code>seguimiento</code>	Doble factor, fugas de datos, constancia de recepción, boleta del comprador	22
<code>peru</code>	Escalones mayoristas, cotización, intenciones de negocio	16
<code>catalogo</code>	Variedad del asesor, relleno de demostración, costo oculto, compresión	14
<code>seguridad</code>	Cabeceras, cuotas, inyección, recorrido de rutas	11
Total	2 361 líneas de pruebas	222

07 FALLAS

Seis fallas que habrían arruinado la demostración

Las encontré probar la tienda como cliente, no leer el código. Van aquí porque explican por qué existen la mitad de las pruebas.

CRÍTICA El asesor entregaba el teléfono de otro cliente

Consultando con un código de pedido, la respuesta traía el teléfono del comprador. Los códigos eran adivinables y no tenían límite de intentos. Se quitó el dato personal de la respuesta y se puso cuota de 30 por minuto.

VENTA PERDIDA Al mayorista no le decía cuántas había

«50 bolsas de maca» devolvía el precio unitario y nada más. Nunca mencionaba que solo quedaban 42. Ahora cotiza con descuento por volumen y dice cuántas entrega hoy y cuántas faltan.

VENTA PERDIDA Pedía bolsas y le cotizaban un frasco

Con 400 productos, «50 bolsas de maca negra» terminaba cotizando un frasco de 60 ml. En una venta al por mayor ese detalle hace desconfiar de todo lo demás. El envase que nombra el cliente ahora manda.

CONTABLE Dos céntimos de diferencia con la boleta

Un pedido de S/ 131.80 emitía un comprobante de S/ 131.78, por redondear dos veces. Se reescribió la aritmética anclando en el importe cobrado.

CONFIANZA El asesor recomendaba lo mismo tres veces

«Me duele la rodilla» devolvía la misma crema en tres presentaciones: una sola opción disfrazada de tres. Ahora las tres recomendaciones son tres productos distintos.

DATOS El RUC de ejemplo era inválido

El RUC que venía en la configuración no pasaba el dígito verificador. Lo detectó el propio validador módulo 11 escrito para el checkout. Corregido, y con prueba que lo vigila.

Lo que falta para que el negocio opere

La demo está completa. Para que el cliente *facture* con esto, falta lo siguiente, ordenado por lo que primero se notaría.

TRÁMITE

El envío a SUNAT no está conectado.

Se emiten boletas y facturas con numeración correcta, XML UBL 2.1 y QR, pero la función de envío está deliberadamente vacía. Necesita certificado digital `.pfx`, usuario SOL secundario y un OSE contratado. Hoy los comprobantes son válidos en pantalla, no ante SUNAT.

DATOS

Los 400 productos son inventados y la mitad de las fotos son de banco libre.

Hay que cargar el catálogo real del negocio —desde el panel, uno por uno, o reemplazando el archivo del catálogo— y sus fotos propias.

TRÁMITE

No hay pasarela de pago.

El pedido se registra, pero el cobro es contra entrega o transferencia. Integrar Yape, Niubiz o Culqi requiere contrato con el proveedor.

CÓDIGO

Corre solo en una laptop.

Para que funcione desde el celular del dueño y el del repartidor hace falta hosting, dominio y HTTPS. Hoy es `localhost`.

CÓDIGO

Un solo usuario.

No hay cuentas separadas para el dueño y para quien despacha, ni permisos distintos entre ellos.

CÓDIGO

Sin respaldo automático

de la base de datos.

Raíz Andina · demostración comercial · Node 22.5+ · sin dependencias externas ·

`npm start` en `localhost:3000`